

Supplementary Material for: Conditioning Matters: Stabilizing Inversion and Attention in Diffusion Image Editing

Appendix

A. Reconstruction Bound

In this appendix, we provide the complete derivation of the inversion–reconstruction error bound stated in the main paper. We begin by introducing global smoothness assumptions on the learned velocity field and then derive the global error bound for Euler discretization of the corresponding ODE flow. We then describe our practical estimator for the local Lipschitz constant based on a memory-efficient stochastic power-iteration scheme using Jacobian–vector (JVP) and vector–Jacobian (VJP) products.

Preliminaries We consider the latent trajectory governed by the flow

$$\dot{\mathbf{x}}(t) = \mathbf{v}(\mathbf{x}(t), t; P), \quad t \in [0, T], \quad (1)$$

where $\mathbf{v}$ is the velocity field conditioned on the textual prompt P . Both inversion (integrating $t : 0 \rightarrow T$) and reconstruction ($T \rightarrow 0$) employ the *same* velocity field and the same discretization scheme. Although flow-based generative models typically employ non-uniform time schedules, each update can be viewed as a first-order Euler integration step with a locally defined step size h .

Assumption A.1 (Global mild smoothness). The velocity field $\mathbf{v}(\mathbf{x}, t; P_{\text{src}})$ is continuously differentiable in both $\mathbf{x}$ and t , and its Jacobian and total time derivative are uniformly bounded over a domain $\Omega \subset \mathbb{R}^d$ that contains all trajectories of interest. That is, there exist constants $L, C > 0$ such that for all $\mathbf{x} \in \Omega$ and $t \in [0, T]$,

$$\|J_v(\mathbf{x}, t)\|_2 \leq L, \quad (2)$$

$$\|J_v(\mathbf{x}, t)\mathbf{v}(\mathbf{x}, t) + \partial_t \mathbf{v}(\mathbf{x}, t)\| \leq 2C. \quad (3)$$

Equivalently,

$$\begin{aligned} L &= \sup_{\mathbf{x} \in \Omega, t \in [0, T]} \|J_v(\mathbf{x}, t)\|_2, \\ C &= \frac{1}{2} \sup_{\mathbf{x} \in \Omega, t \in [0, T]} \|J_v(\mathbf{x}, t)\mathbf{v}(\mathbf{x}, t) + \partial_t \mathbf{v}(\mathbf{x}, t)\|. \end{aligned} \quad (4)$$

Here, L is a global Lipschitz constant of the velocity field, while C controls the magnitude of the second-order term $\frac{d}{dt} \mathbf{v} = J_v \mathbf{v} + \partial_t \mathbf{v}$, which determines the curvature of Euler trajectories.

Inversion–Reconstruction Error Bound Under the smoothness assumptions stated above, the classical global error of explicit Euler integration with step size h satisfies:

$$\|\mathbf{x}_k - \mathbf{x}(t_k)\| \leq \frac{C}{L} (e^{Lt_k} - 1)h. \quad (5)$$

Since both inversion ($0 \rightarrow T$) and reconstruction ($T \rightarrow 0$) integrate the same velocity field using Euler discretization, the total error can be bounded by the sum of the two Euler integration errors.

Proposition 1 (Inversion–Reconstruction Error). *Under the bounds in Eqs. (2)–(3), the reconstruction obtained by integrating forward to T and back satisfies:*

$$\|\mathbf{x}_{\text{rec}}(0; P) - \mathbf{x}_0\| \leq 2 \frac{C}{L} (e^{LT} - 1) h_{\max}. \quad (6)$$

Here, $h_{\max} = \max_k h_k$ denotes the maximum step size along the discretized trajectory. For flow models with $T = 1$, the exponential dependence on L dominates the bound; therefore smaller Lipschitz constants directly lead to tighter reconstruction guarantees.

Estimating the Lipschitz Constant L Direct computation of

$$L = \sup_{\mathbf{x}, t} \|J_v(\mathbf{x}, t)\|_2$$

is infeasible because the Jacobian $J_v \in \mathbb{R}^{d \times d}$ for diffusion models has dimension factor d in the tens of thousands. Forming or storing this matrix is impossible, and computing its spectral norm exactly would require a prohibitively expensive singular value decomposition.

To obtain a tractable approximation, we estimate $\|J_v\|_2$ using *stochastic power iteration*. The spectral norm satisfies

$$\|J_v\|_2 = \max_{\|u\|=1} \|J_v u\|,$$

and repeated application of $J_v^\top J_v$ drives any vector toward the dominant singular vector. Crucially, this requires only Jacobian–vector products (JVPs) and vector–Jacobian products (VJPs), both of which can be computed through automatic differentiation without forming J_v explicitly. This yields an tractable and memory-efficient estimator as demonstrated in Alg. 1, which is applicable to high-dimensional velocity fields.

Trajectory-level estimation. We estimate local Jacobian spectral norms along the inversion trajectory. For every `stride` steps, we run M iterations of power iteration and obtain a local singular value estimate $\hat{\lambda}_k$. To ensure robustness while avoiding pathological outliers caused by rare numerical spikes, we report the empirical Lipschitz constant as the 95th percentile:

$$L_{\text{emp}} = \text{P95}(\{\hat{\lambda}_k\}).$$

In all experiments, we use `stride` = 2 and $M = 3$, which provides a practical balance between estimation accuracy and computational overhead.

Algorithm 1 Stochastic Spectral-Norm Estimation of $\|J_v\|_2$ via JVP/VJP

Require: Inversion trajectory $\{(\mathbf{x}_k, t_k)\}$, stride s , iterations M

- 1: $\mathcal{K} \leftarrow \{k \mid k \equiv 0 \pmod{s}\}$
 - 2: **for** each $k \in \mathcal{K}$ **do**
 - 3: Initialize $u_0 \sim \mathcal{N}(0, I)$ and normalize
 - 4: **for** $m = 0$ **to** $M - 1$ **do**
 - 5: $v_m \leftarrow J_v(\mathbf{x}_k, t_k) u_m$ (JVP)
 - 6: $w_m \leftarrow J_v(\mathbf{x}_k, t_k)^\top v_m$ (VJP)
 - 7: $u_{m+1} \leftarrow w_m / \|w_m\|$
 - 8: **end for**
 - 9: $\hat{\lambda}_k \leftarrow \|J_v(\mathbf{x}_k, t_k) u_M\|$
 - 10: **end for**
 - 11: **return** $L_{\text{emp}} = \text{P95}(\{\hat{\lambda}_k : k \in \mathcal{K}\})$
-

B. Inversion Stability

This experiment examines how conditioning precision affects inversion stability. In this subsection, we describe the experimental procedure used to compute the directional deviation Δ , which measures the sensitivity of the velocity field to local perturbations along the inversion trajectory.

To empirically examine this effect, we construct conditioning signals with progressively richer semantic descriptions by expanding the original prompts with additional image-grounded details. This procedure produces three levels of textual conditioning signals: P_1 (coarse), P_2 (detailed), and P_3 (comprehensive). For each conditioning signal P , we perform the following steps:

1. **Inversion Trajectory Construction:** Given the prompt P and its corresponding source image $\mathbf{x}_0$, we run the forward process in flow matching conditioned on P to obtain the inversion trajectory $\{\mathbf{x}_{\sigma_t}^{\text{inv}}\}_{t=0}^T$ with the velocity field $\{\mathbf{v}_{\sigma_t}(\mathbf{x}_{\sigma_t}^{\text{inv}}, \sigma_t, P)\}_{t=0}^T$ along this trajectory.
2. **Perturbation:** At each timestep t , we generate N perturbed samples $\{\mathbf{x}_{\sigma_t}^{(i)}\}_{i=1}^N$ by adding Gaussian noise:

$$\mathbf{x}_{\sigma_t}^{(i)} = \mathbf{x}_{\sigma_t}^{\text{inv}} + \delta^{(i)}, \quad \delta^{(i)} \sim \mathcal{N}(0, \epsilon^2 \mathbf{I})$$

In this experiment, we set $\epsilon = 0.005$. Similar conclusions can also be drawn under different ϵ settings.

3. **Velocity Prediction:** For each perturbed latent $\mathbf{x}_{\sigma_t}^{(i)}$, we compute its velocity $\mathbf{v}$ conditioned on the signal P :

$$\mathbf{v}_{\sigma_t}^{(i)} = \mathbf{v}_\theta(\mathbf{x}_{\sigma_t}^{(i)}, \sigma_t, P)$$

4. **Angular Deviation Calculation:** To assess the stability of the velocity field, we measure the mean angular deviation Δ between the velocities $\mathbf{v}_{\sigma_t}^{(i)}$ on perturbed latents $\mathbf{x}_{\sigma_t}^{(i)}$ and the reference velocity $\mathbf{v}_{\sigma_t}$ along the inversion trajectory:

$$\Delta = \frac{1}{T} \frac{1}{N} \sum_{t=1}^T \sum_{i=1}^N \left(1 - \cos \left(\mathbf{v}_{\sigma_t}^{(i)}, \mathbf{v}_{\sigma_t} \right) \right),$$

A smaller Δ indicates lower velocity deviation under perturbations and thus reflects a more stable and consistent velocity field.

5. **Statistical Comparison:** For computational efficiency, we conduct this analysis on an official evaluation subset of **PIE-Bench**. For each image, we compute Δ and report the dataset-level average $\overline{\Delta}$.

C. Effectiveness of Cross-Branch Attention Manipulation

In this experiment, we study how conditioning precision and alignment affect the distance between attention maps of the source and target branches during attention manipulation. For each editing instruction, we construct conditioning signals with different levels of structural alignment between the source and target branches. Starting from aligned source–target conditioning pairs, we introduce controlled perturbations to the conditioning structure while preserving the underlying semantic content. This allows us to analyze how conditioning alignment affects attention behavior during editing.

To isolate effects purely attributable to attention behavior—rather than inversion—we directly sample the latent $\mathbf{x}_T$ from a Gaussian distribution as the initialization for source/target branches, instead of performing inversion process.

Attention distance. To measure the consistency between attention features in the source and target branches, we compute the distance between their corresponding attention maps. Here, we provide a detailed explanation of how we compute the distance between attention maps.

In the FLUX model, the attention map is computed as

$$A = \text{softmax}(QK^\top / \sqrt{d}),$$

where the query and key matrices are constructed by concatenating prompt and image features,

$$Q = [Q_p, Q_{\text{img}}], \quad K = [K_p, K_{\text{img}}].$$

This yields a block-structured attention map:

$$A = \begin{bmatrix} Q_p K_p^\top & Q_p K_{\text{img}}^\top \\ Q_{\text{img}} K_p^\top & Q_{\text{img}} K_{\text{img}}^\top \end{bmatrix},$$

where the top-left and bottom-right blocks correspond to self-attention among text tokens and image patches, respectively, and the off-diagonal blocks encode cross-attention between them. We focus on the top-right cross-attention block

$$A^{\text{cross}} = Q_p K_{\text{img}}^\top,$$

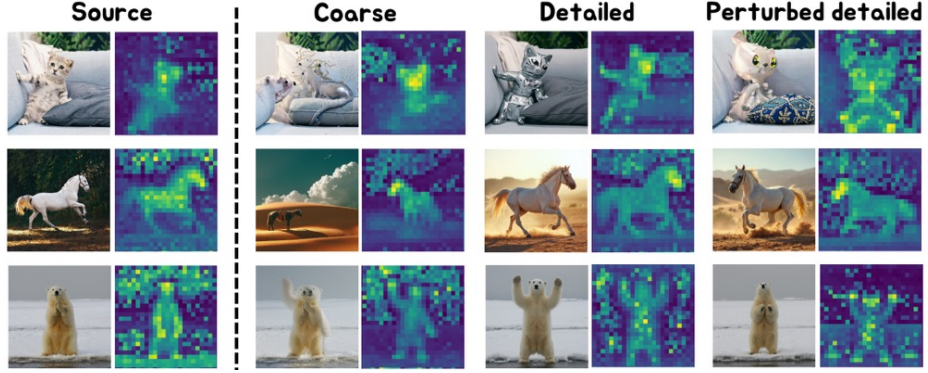


Fig. 1. This figure presents the cross-attention maps of core content—specifically, the words "cat", "horse", and "bear" shown in the three respective rows—in the source branch and the target branch performed on different settings of conditioning signals, along with the corresponding editing results. The figure demonstrates that more precise and aligned textual conditioning lead to more consistent cross-attention maps and more accurate editing outcomes.

which captures how each text token attends to image patches. In our experiment, we quantify attention consistency at the cross-attention block. Concretely, we adopt the following analysis pipeline to measure cross-attention consistency between the source and target branches:

1. **Cross-attention map extraction:** For each layer l , timestep t , and attention head h , we extract the full cross-attention block $A_{l,t,h}^{\text{cross}}$ for both the source and target branches.
2. **Aggregation across layers, timesteps, and heads:** We average the extracted cross-attention maps over all layers, timesteps, and heads to obtain aggregated cross-attention maps

$$\mathbf{A}_{\text{src}}^{\text{cross}} \quad \text{and} \quad \mathbf{A}_{\text{tgt}}^{\text{cross}}$$

for the source and target branches, respectively.

3. **Attention dissimilarity metric:** We compute the (optionally normalized) ℓ_2 distance between the aggregated source and target cross-attention maps:

$$D_{\text{attn}} = \|\mathbf{A}_{\text{src}}^{\text{cross}} - \mathbf{A}_{\text{tgt}}^{\text{cross}}\|_2,$$

where the maps are flattened into vectors before computing the norm.

4. **Statistical comparison:** We evaluate D_{attn} for each example in the dataset and report the average within each experimental group (e.g., different conditioning precision or syntactic variants) for comparison.

In practice, higher attention consistency facilitates more stable and semantically coherent injection of attention features during editing, leading to improved

editing results as shown in Fig. 1. These observations suggest that more precise and better-aligned conditioning signals help improve the effectiveness of attention manipulation in diffusion-based image editing.

D. Implementation Details

Our implementation is based on FLUX.1-dev, with both inversion and editing performed using the default 25 sampling steps. For evaluation on PIE-Bench, attention manipulation is applied at timesteps $[0, 4]$ to balance background preservation and editing fidelity. To ensure reproducibility, we fix the random seed to 2 for all experiments involving stochasticity. All experiments are conducted on two NVIDIA A800 GPUs under CUDA 12.0. The base FLUX.1-dev model requires about 48GB of GPU memory for processing 512×512 images in PIE-Bench. We use Ubuntu 18.04.5 as the operating system, with relevant software libraries Python 3.10, PyTorch 2.5.1+cu121 and diffusers 0.31.0.

Conditioning Refinement

In our implementation, refined conditioning signals are generated using a vision-language model based on the original source-target prompt pairs provided in PIE-Bench. Given the source image and the original prompts, the model produces more detailed textual descriptions of the source image and refined target prompts that expand the editing instruction while remaining semantically consistent with the source content. The refinement step aims to complement parts of the source image that are not explicitly described in the original prompts. In particular, the refined conditioning signals provide additional descriptions of background context, secondary objects and attributes, as well as scene composition and style information that may be omitted in the original conditioning. To improve inversion stability and attention consistency, the generated source and target conditioning signals follow a shared structural format, ensuring that their semantic components remain aligned. This produces more descriptive and better-aligned conditioning signals for inversion-based editing.

For reproducibility, the system prompts used for conditioning generation are included in the released codebase. We also provide the refined prompts for the full PIE-Bench dataset generated using **Gemini-2.5-Pro** with the same system prompts. Fig. 4–9 present examples of original and refined conditioning signals together with the resulting editing outputs. We observe that more descriptive conditioning signals generally improve background preservation and structural consistency in the edited images.

Time Interval for Attention Manipulation

The timestep interval over which attention manipulation is applied has a significant impact on editing behavior. To study this effect, we evaluate several manipulation intervals and report the editing performance under different manipulation

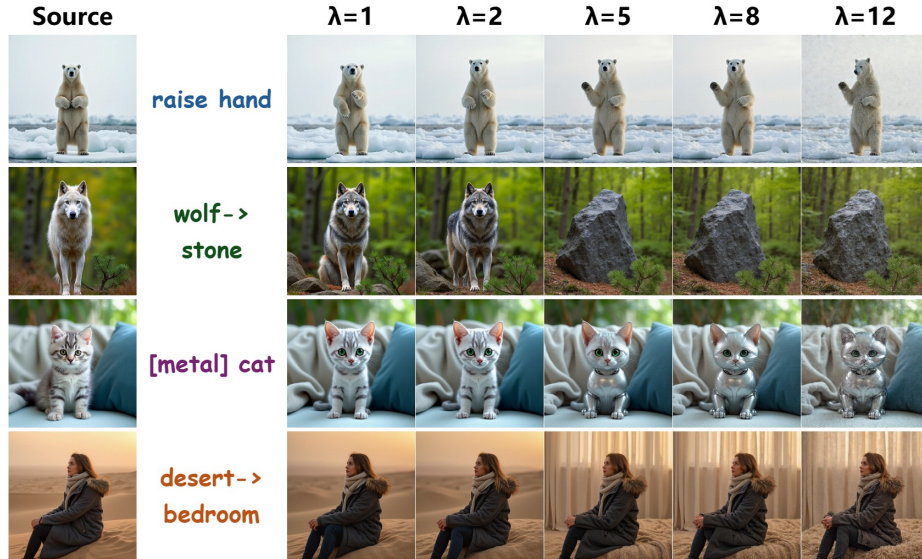


Fig. 2. Qualitative results of editing under different scales of attention reinforcement. A properly chosen scale λ effectively guides the generation of the desired content in the edited output, while an excessively large λ compromises the model’s generative capacity, leading to degraded results.

intervals in Tab. 1. Experiment results demonstrate that as the manipulation interval becomes longer, more structural information from the source branch is injected into the generation process. This leads to improved background preservation, as reflected by better PSNR, SSIM, LPIPS, and MSE scores. However, the stronger structural injection also reduces semantic alignment with the target prompt, as indicated by the decrease in CLIP similarity. This reveals a trade-off between background preservation and editing fidelity. Based on the results in Tab. 1, we adopt the interval $[0 - 4]$, which provides a balanced compromise between structural preservation and semantic editing.

Token-level attention control

Token partition In order to perform accurate token-level attention control, we employ a Longest Common Subsequence (LCS) algorithm to align tokens in source and target prompts, which identifies the edit-relevant tokens k and edit-irrelevant tokens $\bar{k}$. In this section, we provide the complete separating-token algorithm, including backtracking, in Alg. 2. The aligned token indices $\mathcal{K}_{\text{tgt}}$ identify the subset of target prompt tokens that are semantically shared with the source, which we designate as edit-irrelevant tokens $\bar{k}$ for attention replacement. Conversely, the remaining tokens in the target prompt that are not part of the LCS, which are identified as edit-relevant tokens k and serve as the focus of attention reinforcement.

Interval	Structure	Background Preservation				CLIP Similarity		CLIP Similarity*	
	Distance ↓	PSNR ↑	LPIPS ↓	MSE ↓	SSIM ↑	Whole ↑	Edited ↑	Whole ↑	Edited ↑
0-2	37.73	21.27	132.95	108.86	81.92	26.42	23.45	28.54	24.23
0-4	23.20	24.03	98.60	59.61	85.75	26.18	23.07	28.38	23.90
0-6	15.86	26.16	78.18	38.83	88.15	25.48	22.38	27.86	23.41
0-8	11.23	27.85	63.30	27.20	89.96	24.92	21.75	27.42	22.88

Table 1. Effect of manipulation interval on editing performance on PIE-Bench.

Prompt	Mean	Median	P10	P90
Source	90.2%	92.8%	77.2%	99.3%
Target	78.3%	78.9%	65.4%	90.6%

Table 2. Statistics of shared-token coverage rates.

To examine the stability of our LCS-based token matching, we analyze the shared-token coverage between the source and target prompts under LCS alignment. Tab. 2 reports the resulting statistics across the dataset. The average coverage reaches 90.2% for the source prompts and 78.3% for the target prompts, with the P10 coverage remaining above 77.2% and 65.4%, respectively. The slightly lower target coverage is expected, as the refined target prompts contain additional tokens describing and emphasizing newly introduced semantics. This behavior arises because conditioning refinement produces semantically aligned source–target prompt pairs, allowing LCS matching to reliably capture the shared semantic components between them. These results indicate that LCS provides stable token-level alignment on the refined prompts and can robustly separate shared and edit-relevant tokens.

Attention Reinforcement Fig. 2 illustrates the effect of attention reinforcement in the proposed token-wise cross-branch attention control. As shown in Fig. 2, when refined conditioning signals become more descriptive, the increased token complexity may dilute the influence of edit-driving tokens during attention computation. As a result, the generated output may fail to incorporate the intended semantic modification ($\lambda = 1$). In practice, attention reinforcement increases the relative influence of edit-driving tokens in the attention computation, allowing the desired editing semantics to be more effectively injected into the generation process. As λ increases, the effect of attention reinforcement becomes progressively stronger, and the desired content gradually appears in the edited output, demonstrating the effectiveness of the proposed mechanism. However, when λ becomes too large, we observe noticeable degradation in image quality. We attribute this behavior to excessive attention scaling, which causes the attention maps during generation to exceed the stable operating range of the base model and thereby compromises the generative capability of the FLUX backbone. This effect is further amplified in inversion-based editing. Because the

Algorithm 2 Token Alignment via LCS

Input: Target $\hat{P}_{tgt}$, Source $\hat{P}_{src}$, Tokenizer $\mathcal{T}$
Output: Structure-preserving token indices $\bar{k}$ and edit-driving token indices k

```

1: target_id  $\leftarrow \mathcal{T}.\text{encode}(\hat{P}_{tgt})$ 
2: src_id  $\leftarrow \mathcal{T}.\text{encode}(\hat{P}_{src})$ 
3:  $n \leftarrow |\text{target\_id}|$ ,  $m \leftarrow |\text{src\_id}|$ 
4: Initialize  $dp$  as zero matrix of size  $(n+1) \times (m+1)$ 
5: for  $i = 0$  to  $n-1$  do
6:   for  $j = 0$  to  $m-1$  do
7:     if target_id[ $i$ ] = src_id[ $j$ ] then
8:        $dp[i+1][j+1] \leftarrow dp[i][j] + 1$ 
9:     else
10:       $dp[i+1][j+1] \leftarrow \max(dp[i][j+1], dp[i+1][j])$ 
11:    end if
12:  end for
13: end for
14:  $\mathcal{K}_{tgt} \leftarrow []$ ,  $\mathcal{K}_{src} \leftarrow []$ 
15:  $i \leftarrow n$ ,  $j \leftarrow m$ 
16: while  $i > 0$  and  $j > 0$  do
17:   if target_id[ $i-1$ ] = src_id[ $j-1$ ] then
18:     Prepend  $i-1$  to  $\mathcal{K}_{tgt}$ 
19:     Prepend  $j-1$  to  $\mathcal{K}_{src}$ 
20:      $i \leftarrow i-1$ ,  $j \leftarrow j-1$ 
21:   else if  $dp[i-1][j] \geq dp[i][j-1]$  then
22:      $i \leftarrow i-1$ 
23:   else
24:      $j \leftarrow j-1$ 
25:   end if
26: end while
27:  $\bar{k} \leftarrow \mathcal{K}_{tgt}$  {structure-preserving tokens}
28:  $k \leftarrow \{0, \dots, n-1\} \setminus \mathcal{K}_{tgt}$  {edit-driving tokens}
29: return  $\bar{k}, k$ 

```

sampling process starts from inverted latents rather than Gaussian noise, the accumulated inversion errors make the subsequent generation trajectory more sensitive to strong attention perturbations. Consequently, excessive attention reinforcement is more likely to cause visual artifacts under this setting (as illustrated in Fig. 3). Based on these observations, we adopt a relatively moderate reinforcement scale of $\lambda = 3$ in all experiments, which effectively enhances editing semantics while avoiding visual artifacts or image degradation.

E. More qualitative results



Fig. 3. The degraded editing results of inversion-based editing on the PIE-Bench dataset when setting an emphasize scale of $\lambda = 5$ for attention reinforcement.



"Coarse_source_prompt": "two birds sitting on a branch",

"Coarse_target_prompt": "two [origami] birds sitting on a branch",

"Refined_target_prompt": "Two [origami] birds sitting on a branch

Background: A soft-focus, blurry background with abstract patches of green, purple, and pale blue.

Core Object: Two small origami birds [attributes: meticulously crafted from folded paper, with sharp creases, geometric forms, and a matte paper texture in shades of brown and grey] [action/state: perched on a branch, facing one another].

Other Objects: A thick, gnarled tree branch [attributes: with a rough, mossy green texture].

Lighting: Bright, natural sunlight illuminating the scene from the side.

Style: A realistic, high-quality photograph with a very shallow depth of field, creating a strong bokeh effect.

EDITING HIGHLIGHT: The two birds are intricate origami models made of folded paper, featuring crisp edges, sharp creases, and a geometric construction."

Fig. 4. Example 1 of the effectiveness of our conditioning refinement strategy in image editing.

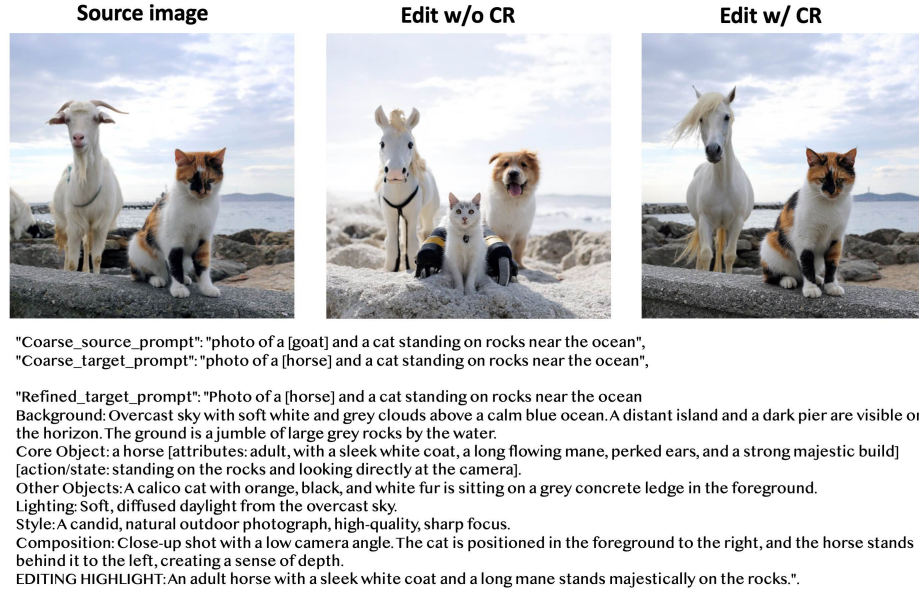


Fig. 5. Example 2 of the effectiveness of our conditioning refinement strategy in image editing.

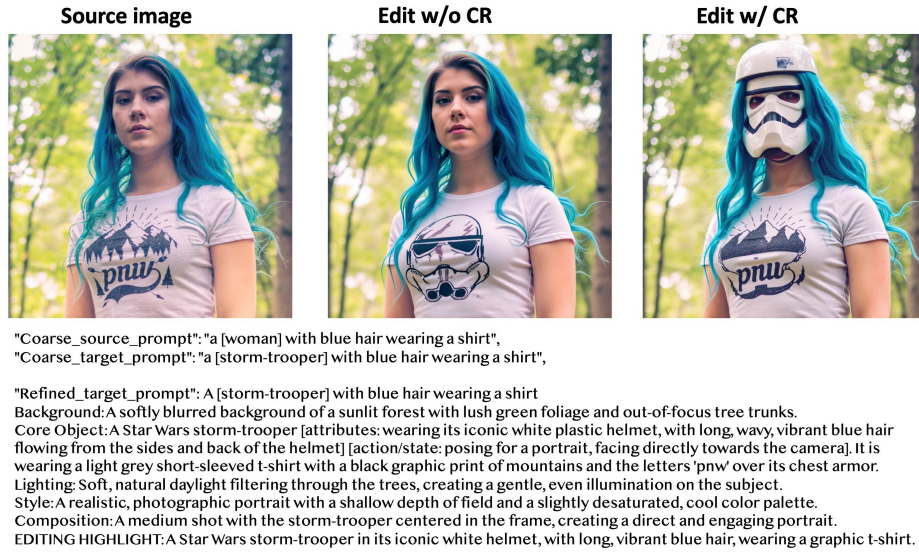


Fig. 6. Example 3 of the effectiveness of our conditioning refinement strategy in image editing.

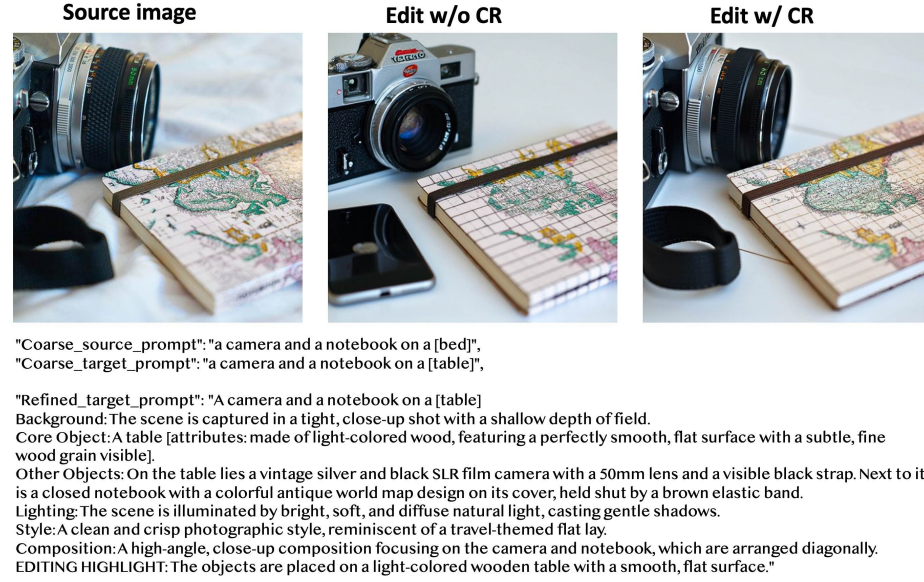


Fig. 7. Example 4 of the effectiveness of our conditioning refinement strategy in image editing.



Fig. 8. Example 5 of the effectiveness of our conditioning refinement strategy in image editing.

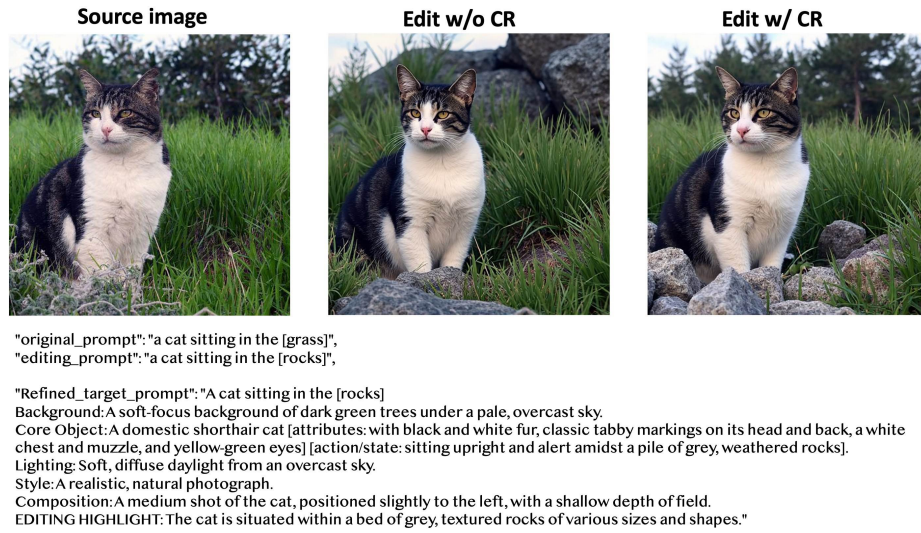


Fig. 9. Example 6 of the effectiveness of our conditioning refinement strategy in image editing.